

© Joshua M. Willman 2020

J. M. Willman, *Beginning PyQt*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_12

12. Extra Projects

Joshua M. Willman¹

(1) Hampton, VA, USA

This book has tried to take a practical approach to creating GUIs. As you use PyQt5 and Python more and more, you will find yourself learning about other modules and classes that you will need in your applications. Each chapter set up an idea and worked hard to break those projects down into their fundamental parts so that you could learn new ideas along the way.

PyQt5 has quite a few modules for a variety of purposes, and the chapters in this book only scratched the surface of the many possibilities for designing GUIs.

In Chapter [12](#), we will take a look at a few extra examples to give you ideas for other projects or to help you in creating new types of user interfaces. These projects will not go into great lengths of detail, but rather focus on explaining the key points of each new program and leave it up to you to research the details that you are unsure about, either by finding the answers in a different chapter or by searching online for help.

The projects in this chapter will take a look at the following concepts:

- Displaying directories and files using the `QFileSystemModel` class
- Working with multiple-document interface (MDI) applications and the `QCamera` class
- Creating a simple clock GUI with `QDate` and `QTime`
- Exploring the `QCalendarWidget` class
- Building Hangman with `QPainter` and other PyQt classes
- Building the framework for a web browser using the `QtWebEngineWidgets` module

Project 12.1 – Directory Viewer GUI

For every operating system, there needs to be some method for a user to access the data and files located in it. These files are stored in a hierarchical file system, displaying drives, directories, and files in groups so that you only view the files that you are interested in seeing.

Whether you use a command-line interface or a graphical user interface, there needs to be some way to create, remove, and rename files and directories. However, if you are already interacting with one interface, it may be more convenient to locate files or directories that you need in

your current application rather than opening new windows or other programs.

This project shows you how to set up an interface for viewing the files on your local system. There are two key classes that will be introduced in this project – **QFileSystemModel**, which grants you access to the file system on your computer, and **QTreeView**, which provides a visual representation of data using a tree-like structure (Listing 12-1).

```
# display_directory.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QFileSystemModel, QTreeView, QFrame, QAction, QMenu)

class DisplayDirectory(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()

    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setMinimumSize(500, 400)
        self.setWindowTitle('12.1 - View Directory GUI')
        self.createMenu()
        self.setupTree()
        self.show()

    def createMenu(self):
        """
        Set up the menu bar.
        """
        open_dir_act = QAction('Open Directory...', self)
        open_dir_act.triggered.connect(self.chooseDirectory)
        root_act = QAction("Return to Root", self)
        root_act.triggered.connect(self.returnToRootDirectory)
        # Create menubar
        menu_bar = self.menuBar()
        #menu_bar.setNativeMenuBar(False) # Uncomment for MacOS
        # Create file menu and add actions
        dir_menu = menu_bar.addMenu('Directories')
        dir_menu.addAction(open_dir_act)
        dir_menu.addAction(root_act)

    def setupTree(self):
        """
        Set up the QTreeView so that it displays the contents
        of the local filesystem.
        """
        self.model = QFileSystemModel()
        self.model.setRootPath('')
        self.tree = QTreeView()
        self.tree.setIndentation(10) # Indentation of items
        self.tree.setModel(self.model)
        # Set up container and layout
        frame = QFrame()
        frame_v_box = QVBoxLayout()
        frame_v_box.addWidget(self.tree)
        frame.setLayout(frame_v_box)
        self.setCentralWidget(frame)

    def chooseDirectory(self):
        """
        Select a directory to display.
        """
```

```

"""
file_dialog = QFileDialog(self)
file_dialog.setFileMode(QFileDialog.Directory)
directory = file_dialog.getExistingDirectory(self, "Open Directory", "", QFileDialog.ShowDirsOn
self.tree.setRootIndex(self.model.index(directory))
def returnToRootDirectory(self):
"""
Re-display the contents of the root directory.
"""
self.tree.setRootIndex(self.model.index(''))
if __name__ == '__main__':
app = QApplication(sys.argv)
window = DisplayDirectory()
sys.exit(app.exec_())

```

Listing 12-1 Code for directory viewer GUI

The directory viewer application can be seen in Figure [12-1](#).

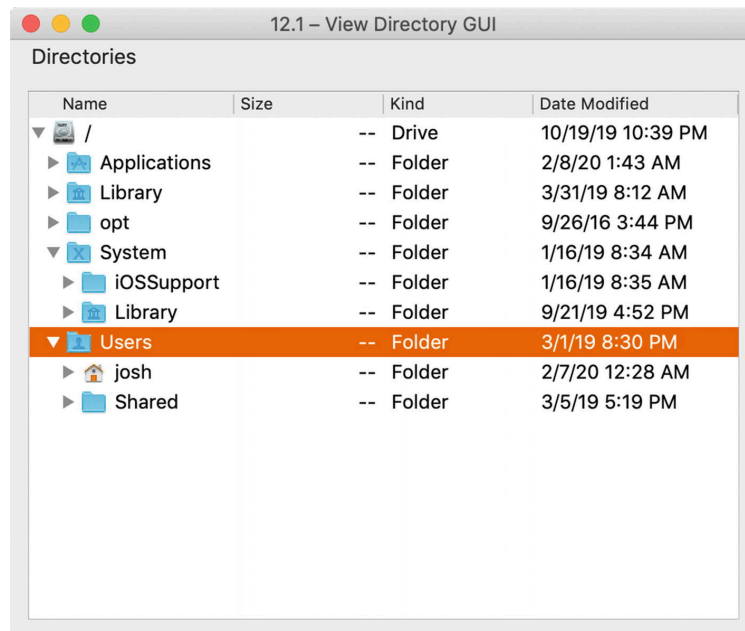


Figure 12-1 Directory viewer displaying the local system's directories

Explanation

Begin by importing the necessary modules for this GUI. For this project, we will need to use the model/view paradigm to view the data on your computer. For more information about model/view programming, refer to Chapter [10](#).

The `QFileSystemModel` class provides the model we need to access data on the local file system. While not included in this project, you could also use `QFileSystemModel` to rename or remove files and directories, create new directories, or use it with other display widgets as part of a browser.

The `QTreeView` class will be used to display the contents of the model in a hierarchical tree view.

For this GUI, we will create a `Directories` menu with actions that will either let the user view a specific directory or return back to the root directory. The menu system can be seen in Figure [12-2](#).

Create an instance of the `QFileSystemModel` class, `model`, and set the directory to the root path on your system.

```
self.model.setRootPath('') # Sets path to system's root path
```

Set the model for the `tree` object to show the contents of the file system using `setModel()`. To choose a different directory, the user can select `Open Directory...` from the menu and a file dialog will appear. A new directory can then be selected and set as the new root path to be displayed in the `tree` object.

```
self.tree.setRootIndex(self.model.index(directory))
```

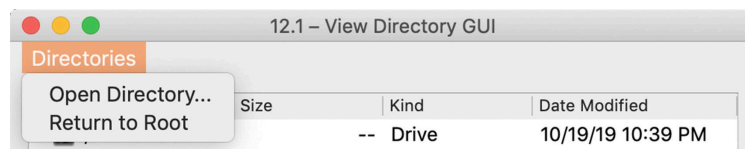


Figure 12-2 The menu for the directory viewer GUI

Project 12.2 – Camera GUI

When creating GUIs, there are a number of ways to tackle the issue of interfaces with multiple windows. You could use stacked or tabbed widgets, but these methods only allow for one window to be displayed at a time. Another option is to use dock widgets and allow windows to be floatable or used as secondary windows.

For this project, you will see how to set up a multiple-windowed GUI using the **QMdiArea** class. **QMdiArea** provides the area for displaying MDI windows. Multiple-document interface (MDI) is a type of interface that allows users to work with multiple windows at the same time. MDI applications require less memory resources and make the process of laying out subwindows much simpler.

Let's take a look at how to use the **QCamera** class to create an MDI application (Listing [12-2](#)).

```
# camera.py
# Import necessary modules
import os, sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QListWidget, QListWidgetItem, QLabel, QGroupBox)
from PyQt5.QtMultimedia import QCamera, QCameraInfo, QCameraImageCapture
from PyQt5.QtMultimediaWidgets import QCameraViewfinder
from PyQt5.QtCore import Qt
class Camera(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 600, 400)
        self.setWindowTitle('12.2 - Camera GUI')
```

```

self.setupWindows()
self.show()
def setupWindows(self):
    """
    Set up QMdiArea parent and subwindows.
    Add available cameras on local system as items to
    list widget.
    """
    # Create images directory if it does not already exist
    path = 'images'
    if not os.path.exists(path):
        os.makedirs(path)
    # Set up list widget that will display identified
    # cameras on your computer.
    picture_label = QLabel("Press 'Spacebar' to take pictures.")
    camera_label = QLabel("Available Cameras")
    self.camera_list_widget = QListWidget()
    self.camera_list_widget.setAlternatingRowColors(True)
    # Add availableCameras to a list to be displayed in
    # list widget. Use QCameraInfo() to list available cameras.
    self.cameras = list(QCameraInfo().availableCameras())
    for camera in self.cameras:
        self.list_item = QListWidgetItem()
        self.list_item.setText(camera.deviceName())
        self.camera_list_widget.addItem(self.list_item)
    # Create button that will allow user to select camera
    choose_cam_button = QPushButton("Select Camera")
    choose_cam_button.clicked.connect(self.selectCamera)
    # Create child widgets and layout for camera controls subwindow
    controls_gbox = QGroupBox()
    controls_gbox.setTitle("Camera Controls")
    v_box = QVBoxLayout()
    v_box.addWidget(picture_label, alignment=Qt.AlignCenter)
    v_box.addWidget(camera_label)
    v_box.addWidget(self.camera_list_widget)
    v_box.addWidget(choose_cam_button)
    controls_gbox.setLayout(v_box)
    controls_sub_window = QMdiSubWindow()
    controls_sub_window.setWidget(controls_gbox)
    controls_sub_window.setAttribute(Qt.WA_DeleteOnClose)
    # Create viewfinder subwindow
    self.view_finder_window = QMdiSubWindow()
    self.view_finder_window.setWindowTitle("Camera View")
    self.view_finder_window.setAttribute(Qt.WA_DeleteOnClose)
    # Create QMdiArea widget to manage subwindows
    mdi_area = QMdiArea()
    mdi_area.tileSubWindows()
    mdi_area.addSubWindow(self.view_finder_window)
    mdi_area.addSubWindow(controls_sub_window)
    # Set mdi_area widget as the central widget of main window
    self.setCentralWidget(mdi_area)
def setupCamera(self, cam_name):
    """
    Create and setup camera functions.
    """
    for camera in self.cameras:
        # Select camera by matching cam_name to one of the
        # devices in the cameras list.
        if camera.deviceName() == cam_name:
            self.cam = QCamera(camera) # Construct QCamera device
            # Create camera viewfinder widget and add it to the view_finder_window.

```

```

        self.view_finder = QCameraViewfinder()
        self.view_finder_window.setWidget(self.view_finder)
        self.view_finder.show()
        # Sets the viewfinder to display video
        self.cam.setViewfinder(self.view_finder)
        # QCameraImageCapture() is used for taking
        # images or recordings.
        self.image_capture = QCameraImageCapture(self.cam)
        # Configure the camera to capture still images.
        self.cam.setCaptureMode(QCamera.CaptureStillImage)
        self.cam.start() # Slot to start the camera
    else:
        pass
def selectCamera(self):
    """
    Slot for selecting one of the available cameras displayed in list widget.
    """
    try:
        if self.list_item.isSelected():
            camera_name = self.list_item.text()
            self.setupCamera(camera_name)
        else:
            print("No camera selected.")
            pass
    except:
        print("No cameras detected.")
def keyPressEvent(self, event):
    """
    Handle the key press event so that the camera takes images.
    """
    if event.key() == Qt.Key_Space:
        try:
            self.cam.searchAndLock()
            self.image_capture.capture("images/")
            self.cam.unlock()
        except:
            print("No camera in viewfinder.")
# Run program
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = Camera()
    sys.exit(app.exec_())

```

Listing 12-2 Example code to show how to create MDI applications

Your GUI should look similar to the one in Figure [12-3](#).

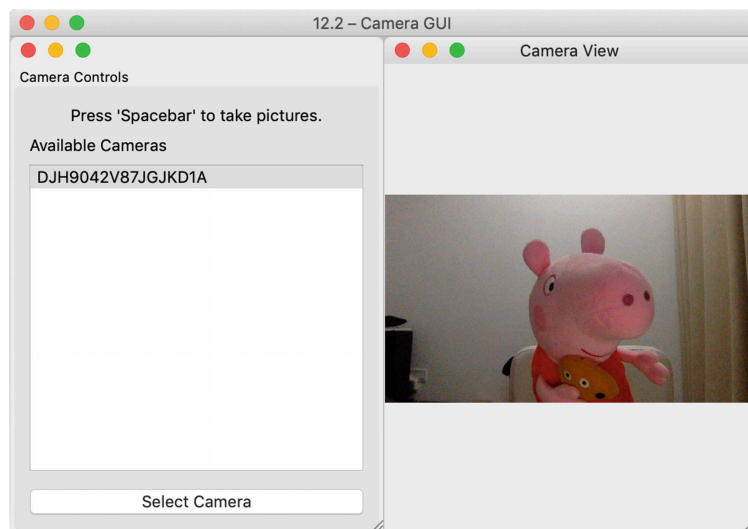


Figure 12-3 Camera GUI is composed of multiple windows that allow the user to select available cameras and view the camera's viewfinder

Explanation

For this project, we are going to import some new classes. From the `QtWidgets` module, the `QMdiArea` and `QMdiSubWindow` classes are used to create the MDI windows.

The **QtMultimedia** module provides access to a number of multimedia tools including audio, video, and camera capabilities. The `QCamera` class provides the interface to work with camera devices. `QCameraInfo` supplies information about available cameras. `QCameraImageCapture` is used for recording media.

From the **QtMultimediaWidgets** module, the `QCameraViewfinder` class sets up the camera viewfinder widget. In photography, the viewfinder is used for focusing and viewing the subject being photographed.

This application contains two subwindows, one for displaying the viewfinder and the other for listing the available cameras that you can choose from in a `QListWidget` object. In the `setupWindows()` method, the labels, list widget, and push button are arranged inside of a `QGroupBox` widget. The user can select a camera from the list. The `Select Camera` button emits a signal that is connected to the `selectCamera()` slot. Next, the `QMdiArea` object, `mdi_area`, that is used as a container for the subwindows is created. This will be the central widget for the main window.

Child windows are instances of `QMdiSubWindow`. The subwindows inside of `mdi_area` are created in relation to each other. In this project, they are arranged as tiles using `tileSubWindows()`. Another option is to lay them out using a cascaded style.

```
mdi_area.cascadeSubWindows()
```

TipA menubar could also be added to the main window that controls the subwindows. For example, subwindows could be set as checkable in or-

der to close or reopen them. Or a menu item could allow the user to switch between tiled or cascaded windows.

If the user clicks the push button and an available camera is selected, then the `setupCamera()` method is called. Refer to the comments in the code to learn how to set up the viewfinder. This method is adapted from the Qt document web site.¹

Using `QCameraImageCapture()`, the user is also able to take pictures of the viewfinder. Image capturing is handled by the `keyPressEvent()`. When the spacebar is pressed, a picture is taken and saved to the "images/" folder. The folder will be created if it does not already exist.

Project 12.3 – Simple Clock GUI

PyQt5 also provides classes for dealing with dates, **QDate**, or time, **QTime**. The **QDateTime** class supplies functions for working with both dates and time. All three of these classes include methods for handling time-related features.

Let's take a brief look at the **QDateTime** class. The following snippet of code creates an instance of **QDateTime** that returns the current date and time using the `currentDateTime()` method:

```
date_time = QDateTime.currentDateTime()
print(date_time.toString(Qt.DefaultLocaleLongDate))
```

The current date and time is printed to the screen with the following format (set using `Qt.DefaultLocaleLongDate`):

```
February 15, 2020 2:32:31 PM CST
```

There are also other formats, including shorter formats, ISO 8601 format, or UTC format. The `toString()` method returns the date and time as a string. **QDateTime** also handles daylight saving time, different time zones, and the manipulation of times and dates such as adding or subtracting months, days, or hours.

If you only need to work with the individual dates and times, **QDate** and **QTime** also provide similar functions as you shall see in Listing [12-3](#).

```
# clock.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel,
                             QVBoxLayout)
from PyQt5.QtCore import Qt, QDate, QTime, QTimer
class DisplayTime(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
```



```

self.setGeometry(100, 100, 250, 100)
self.setWindowTitle('12.3 - QDateTime Example')
self.setStyleSheet("background-color: black")
self.setupWidgets()
# Create timer object
timer = QTimer(self)
timer.timeout.connect(self.updateDateTime)
timer.start(1000)
self.show()
def setupWidgets(self):
    """
    Set up labels that will display current date and time.
    """
    current_date, current_time = self.getDateTime()
    self.date_label = QLabel(current_date)
    self.date_label.setStyleSheet("color: white; font: 16px Courier")
    self.time_label = QLabel(current_time)
    self.time_label.setStyleSheet("""color: white;
                                   border-color: white;
                                   border-width: 2px;
                                   border-style: solid;
                                   border-radius: 4px;
                                   padding: 10px;
                                   font: bold 24px Courier""")
    # Create layout and add widgets

    v_box = QVBoxLayout()
    v_box.addWidget(self.date_label, alignment=Qt.AlignCenter)
    v_box.addWidget(self.time_label, alignment=Qt.AlignCenter)
    self.setLayout(v_box)
def getDateTime(self):
    """
    Returns current date and time.
    """
    date = QDate.currentDate().toString(Qt.DefaultLocaleLongDate)
    time = QTime.currentTime().toString("hh:mm:ss AP")
    return date, time
def updateDateTime(self):
    """
    Slot that updates date and time values.
    """
    date = QDate.currentDate().toString(Qt.DefaultLocaleLongDate)
    time = QTime.currentTime().toString("hh:mm:ss AP")
    self.date_label.setText(date)
    self.time_label.setText(time)
    return date, time
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = DisplayTime()
    sys.exit(app.exec_())

```

Listing 12-3 Code for the clock GUI

The clock application can be seen in Figure [12-4](#).

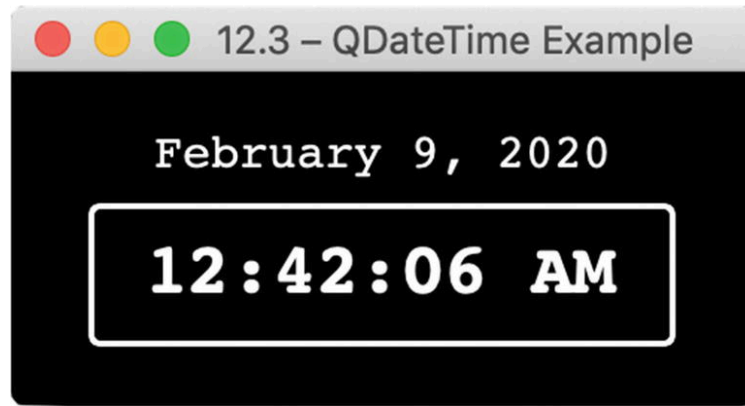


Figure 12-4 The clock GUI displaying the current calendar date and clock time

Explanation

Start by importing the necessary modules, including `QDate`, `QTime`, and `QTimer`, from the `QtCore` module. The `QTimer` class will be used to create a timer object to keep track of the time that has passed and update the labels that hold the date and time accordingly. The timer is set up in `initializeUI()`, and its `timeout()` signal is connected to the `updateDateTime()` slot. The `timeout()` signal is emitted every second.

In order to get the current date and time, the values are retrieved using the `currentDate()` and `currentTime()` methods in the `getDateTime()` method. These are then returned and set as the `current_date` and `current_time`.

```
current_date, current_time = self.getDateTime()
```

While the date is set to use the `Qt.DefaultLocaleLongDate` format, the time uses a sequence of characters to create a format string that displays hours (hh), minutes (mm), seconds (ss), and AM or PM (AP).

```
time = QTime.currentTime().toString("hh:mm:ss AP")
```

The labels that will display the date and time are then instantiated, styled, and added to the layout. The values of the labels are updated using the `updateDateTime()` method.

Project 12.4 – Calendar GUI

This project takes a look at how to set up the `QCalendarWidget` class and use a few of its functions. PyQt makes adding a monthly calendar to your applications rather effortless. The code is provided in Listing [12-4](#) and the calendar can be seen in Figure [12-5](#).

The `QCalendarWidget` class provides a calendar that already has a number of other useful widgets and functions built-in. For example, the calendar already includes a horizontal header that includes widgets for changing the month and the year and a vertical header that displays the week number. The class also includes signals that are emitted whenever the dates, months, and years on the calendar are changed.

The **QDateEdit** widget is used in this application to restrict the values a user can select to within a certain range, specified by minimum and maximum values.

```
# calendar.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QLabel, QCalendarWidget, QDateEdit, QGroupBox, QHBox)
from PyQt5.QtCore import Qt, QDate
from PyQt5.QtGui import QFont
style_sheet = """
    QLabel{
        padding: 5px;
        font: 18px
    }
    QLabel#DateSelected{
        font: 24px
    }
    QGroupBox{
        border: 2px solid gray;
        border-radius: 5px;
        margin-top: 1ex;
        font: 14px
    }
"""

class CalendarGUI(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()

    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setMinimumSize(500, 400)
        self.setWindowTitle('12.4 - Calendar GUI')
        self.createCalendar()
        self.show()

    def createCalendar(self):
        """
        Set up calendar, others widgets and layouts for main window.
        """
        self.calendar = QCalendarWidget()
        self.calendar.setGridVisible(True)
        self.calendar.setMinimumDate(QDate(1900, 1, 1))
        self.calendar.setMaximumDate(QDate(2200, 1, 1))
        # Connect to newDateSelection slot when currently selected date is changed
        self.calendar.selectionChanged.connect(self.newDateSelection)
        current = QDate.currentDate().toString(Qt.DefaultLocaleLongDate)
        self.current_label = QLabel(current)
        self.current_label.setObjectName("DateSelected")
        # Create current, minimum, and maximum QDateEdit widgets
        min_date_label = QLabel("Minimum Date:")
        self.min_date_edit = QDateEdit()
        self.min_date_edit.setDisplayFormat("MMM d yyyy")
        self.min_date_edit.setDateRange(self.calendar.minimumDate(), self.calendar.maximumDate())
        self.min_date_edit.setDate(self.calendar.minimumDate())
        self.min_date_edit.dateChanged.connect(self.minDatedChanged)
        current_date_label = QLabel("Current Date:")
        self.current_date_edit = QDateEdit()
        self.current_date_edit.setDisplayFormat("MMM d yyyy")
        self.current_date_edit.setDate(self.calendar.selectedDate())
```

```

self.current_date_edit.setDateRange(self.calendar.minimumDate(), self.calendar.maximumDate())
self.current_date_edit.dateChanged.connect(self.selectionDateChanged)
max_date_label = QLabel("Maximum Date:")
self.max_date_edit = QDateEdit()
self.max_date_edit.setDisplayFormat("MMM d yyyy")
self.max_date_edit.setDateRange(self.calendar.minimumDate(), self.calendar.maximumDate())
self.max_date_edit.setDate(self.calendar.maximumDate())
self.max_date_edit.dateChanged.connect(self.maxDatedChanged)
# Add widgets to group box and add to grid layout
dates_gb = QGroupBox("Set Dates")
dates_grid = QGridLayout()
dates_grid.addWidget(self.current_label, 0, 0, 1, 2, Qt.AlignAbsolute)
dates_grid.addWidget(min_date_label, 1, 0)
dates_grid.addWidget(self.min_date_edit, 1, 1)
dates_grid.addWidget(current_date_label, 2, 0)
dates_grid.addWidget(self.current_date_edit, 2, 1)
dates_grid.addWidget(max_date_label, 3, 0)
dates_grid.addWidget(self.max_date_edit, 3, 1)
dates_gb.setLayout(dates_grid)
# Create and set main window's layout
main_h_box = QHBoxLayout()
main_h_box.addWidget(self.calendar)
main_h_box.addWidget(dates_gb)
self.setLayout(main_h_box)
def selectionDateChanged(self, date):
    """
    Update the current_date_edit when the calendar's selected date changes.
    """
    self.calendar.setSelectedDate(date)
def minDatedChanged(self, date):
    """
    Update the calendar's minimum date.
    Update max_date_edit to avoid conflicts with maximum and minimum dates.
    """
    self.calendar.setMinimumDate(date)
    self.max_date_edit.setDate(self.calendar.maximumDate())
def maxDatedChanged(self, date):
    """
    Update the calendar's maximum date.
    Update min_date_edit to avoid conflicts with minimum and maximum dates.
    """
    self.calendar.setMaximumDate(date)
    self.min_date_edit.setDate(self.calendar.minimumDate())
def newDateSelection(self):
    """
    Update date in current_label and current_date_edit widgets when user selects a new date.
    """
    date = self.calendar.selectedDate().toString(Qt.DefaultLocaleLongDate)
    self.current_date_edit.setDate(self.calendar.selectedDate())
    self.current_label.setText(date)
if __name__ == '__main__':
    app = QApplication(sys.argv)
    app.setStyleSheet(style_sheet)
    window = CalendarGUI()
    sys.exit(app.exec_())

```

Listing 12-4 The calendar GUI code

The look of your calendar will greatly depend upon the platform that you are using to run the application. An example of the calendar on MacOS can be seen in Figure [12-5](#).

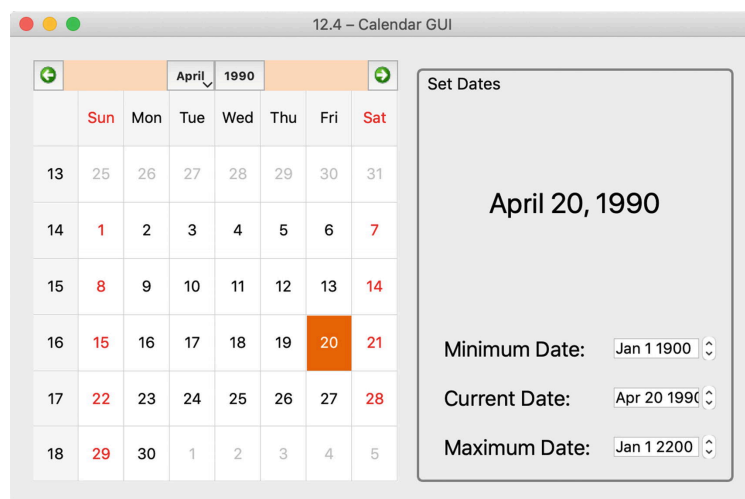


Figure 12-5 The calendar GUI that displays the calendar, the current date, and the widgets that allow the user to search for dates within a specified time range

Explanation

After importing the modules needed for the calendar GUI, the styles for the `QLabel` and `QGroupBox` widgets are prepared using `style_sheet`.

Creating an instance of `QCalendarWidget` is very simple.

```
self.calendar = QCalendarWidget()
```

Next, we set a few of the `calendar` object's parameters. Setting `setGridVisible()` to `True` will make the grid lines visible. In order to specify the date range that a user can select in the calendar, we set the minimum and maximum date values.

```
self.calendar.setMinimumDate(QDate(1900, 1, 1))
self.calendar.setMaximumDate(QDate(2200, 1, 1))
```

Whenever a date is selected in the `calendar` widget, it emits a `selectionChanged()` signal. This signal is connected to the `newDateSelection()` slot that updates the date on the `current_label` and in the `current_date_edit` widget. Selecting a value in the `current_date_edit` widget will also change the other values.

The `QCalendarWidget` class also has a number of functions that allow you to configure its behaviors and appearance. For this project, we create three `QDateEdit` widgets that will allow the user to change the minimum and maximum values for the date range, as well as the current date selected in the calendar.

A displayed format for the date in the `QDateEdit` widget can be set using `setDisplayFormat()`. The date edit objects are also given a date range using `setDateRange()`. The following code is an example of how to set the `min_date_edit` widget's date range by using ranges set earlier for the `calendar` object:

```
self.min_date_edit.setDateRange(self.calendar.minimumDate(), self.calendar.maximumDate())
```

When a date is changed in a date edit widget, it generates a `date-`

`Changed()` signal. Each one of the `QDateEdit` widgets is connected to a corresponding slot that will update the calendar's minimum, maximum, or current date values depending upon which date edit widget is changed. The method for changing the dates is adapted from the Qt document web site.²

Finally, the label and date edit widgets are arranged in a `QGroupBox`.

Project 12.5 – Hangman GUI

PyQt can be used to create a variety of different kinds of applications. Throughout this book, we have looked at quite a few ideas for building GUIs. For this next project, we will be taking a look at how to use `QPainter` and a few other classes to build a game – Hangman. While Hangman is a simple game to play, it can be used to teach a few of the fundamental concepts for using PyQt to create games. The code is presented in Listing 12-5 and the interface can be seen in Figure 12-6.

For this application, the player can select from one of the twenty-six English letters to guess a letter in an unknown word. As each letter is chosen, they will become disabled in the window. If the letter is correct, it will be revealed to the player. Otherwise, a part of the hangman figure's body is drawn on the screen. If all of the letters are correctly guessed, then the player wins. There are a total of six turns.

Whether or not the player wins or loses, a dialog will be displayed to inform the player and allow them to quit or to continue playing.

```
# hangman.py
# Import necessary modules
import sys, random
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QPushButton, QLabel, QFrame, QButtonGr
from PyQt5.QtCore import Qt, QRect, QLine
from PyQt5.QtGui import QPainter, QPen, QBrush, QColor
style_sheet = """
    QWidget{
        background-color: #FFFFFF
    }
    QLabel#Word{
        font: bold 20px;
        qproperty-alignment: AlignCenter
    }
    QPushButton#Letters{
        background-color: #1FAEDE;
        color: #D2DDE1;
        border-style: solid;
        border-radius: 3px;
        border-color: #38454A;
        font: 28px
    }
    QPushButton#Letters:pressed{
        background-color: #C86354;
        border-radius: 4px;
        padding: 6px;
        color: #DFD8D7
    }
```

```

    }
    QPushButton#Letters:disabled{
        background-color: #BBC7CB
    }
    """
# The hangman is drawn on a QLabel object, rather than
# on the main window. This class handles the drawing.
class DrawingLabel(QLabel):
    def __init__(self, parent):
        super().__init__(parent)
        # Variables for positioning drawings
        self.height = 200
        self.width = 300
        # Variables used to keep track of incorrect guesses
        self.incorrect_letter = False
        self.incorrect_turns = 0
        # List to store body parts
        self.part_list = []
    def drawHangmanBackground(self, painter):
        """
        Draw the gallows.
        """
        painter.setBrush(QBrush(QColor("#000000")))
        # drawRect(x, y, width, height)
        painter.drawRect((self.width / 2) - 40, self.height, 150, 4)
        painter.drawRect(self.width / 2, 0, 4, 200)
        painter.drawRect(self.width / 2, 0, 60, 4)
        painter.drawRect((self.width / 2) + 60, 0, 4, 40)
    def drawHangmanBody(self, painter):
        """
        Create and draw body parts for hangman.
        """
        if "head" in self.part_list:
            head = QRect((self.width / 2) + 42, 40, 40, 40)
            painter.setPen(QPen(QColor("#000000"), 3))
            painter.setBrush(QBrush(QColor("#FFFFFF")))
            painter.drawEllipse(head)
        if "body" in self.part_list:
            body = QRect((self.width / 2) + 60, 80, 2, 55)
            painter.setBrush(QBrush(QColor("#000000")))
            painter.drawRect(body)
        if "right_arm" in self.part_list:
            right_arm = QLine((self.width / 2) + 60, 85,
                              (self.width / 2) + 50, (self.height / 2) + 30)
            pen = QPen(Qt.black, 3, Qt.SolidLine)
            painter.setPen(pen)
            painter.drawLine(right_arm)
        if "left_arm" in self.part_list:
            left_arm = QLine((self.width / 2) + 62, 85,
                              (self.width / 2) + 72, (self.height / 2) + 30)
            painter.drawLine(left_arm)
        if "right_leg" in self.part_list:
            right_leg = QLine((self.width / 2) + 60, 135,
                              (self.width / 2) + 50, (self.height / 2) + 75)
            painter.drawLine(right_leg)
        if "left_leg" in self.part_list:
            left_leg = QLine((self.width / 2) + 62, 135,
                              (self.width / 2) + 72, (self.height / 2) + 75)
            painter.drawLine(left_leg)
        # Reset variable
        self.incorrect_letter = False

```

```

def paintEvent(self, event):
    """
    Construct the QPainter and handle painting events.
    """
    painter = QPainter(self)
    self.drawHangmanBackground(painter)
    if self.incorrect_letter == True:
        self.drawHangmanBody(painter)
    painter.end()

class Hangman(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setFixedSize(400, 500)
        self.setWindowTitle('12.5 - Hangman GUI')
        self.newGame()
        self.show()
    def newGame(self):
        """
        Create new Hangman game.
        """
        self.setupHangmanBoard()
        self.setupWord()
        self.setupBoard()
    def setupHangmanBoard(self):
        """
        Set up label object to display hangman.
        """
        self.hangman_label = DrawingLabel(self)
        self.hangman_label.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
    def setupWord(self):
        """
        Open words file and choose random word.
        Create labels that will display '_' depending
        upon length of word.
        """
        words = self.openFile()
        self.chosen_word = random.choice(words).upper()
        #print(self.chosen_word)
        # Keep track of correct guesses
        self.correct_counter = 0
        # Keep track of label objects.
        # Is used for updating the text on the labels
        self.labels = []
        word_h_box = QHBoxLayout()
        for letter in self.chosen_word:
            self.letter_label = QLabel("__")
            self.labels.append(self.letter_label)
            self.letter_label.setObjectName("Word")
            word_h_box.addWidget(self.letter_label)
        self.word_frame = QFrame()
        self.word_frame.setLayout(word_h_box)
    def setupBoard(self):
        """
        Set up objects and layouts for keyboard and main window.
        """
        top_row_list = ["A", "B", "C", "D", "E", "F", "G", "H"]

```



```

mid_row_list = ["I", "J", "K", "L", "M", "N", "O", "P", "Q"]
bot_row_list = ["R", "S", "T", "U", "V", "W", "X", "Y", "Z"]
# Create button group to keep track of letters
self.keyboard_bg = QButtonGroup()
# Set up keys in the top row
top_row_h_box = QHBoxLayout()
for letter in top_row_list:
    button = QPushButton(letter)
    button.setObjectName("Letters")
    top_row_h_box.addWidget(button)
    self.keyboard_bg.addButton(button)
top_frame = QFrame()
top_frame.setLayout(top_row_h_box)
# Set up keys in the middle row
mid_row_h_box = QHBoxLayout()
for letter in mid_row_list:
    button = QPushButton(letter)
    button.setObjectName("Letters")
    mid_row_h_box.addWidget(button)
    self.keyboard_bg.addButton(button)
mid_frame = QFrame()
mid_frame.setLayout(mid_row_h_box)
# Set up keys in the bottom row
bot_row_h_box = QHBoxLayout()
for letter in bot_row_list:
    button = QPushButton(letter)
    button.setObjectName("Letters")
    bot_row_h_box.addWidget(button)
    self.keyboard_bg.addButton(button)
bot_frame = QFrame()
bot_frame.setLayout(bot_row_h_box)
# Connect buttons in button group to slot
self.keyboard_bg.buttonClicked.connect(self.buttonPushed)
keyboard_v_box = QVBoxLayout()
keyboard_v_box.addWidget(top_frame)
keyboard_v_box.addWidget(mid_frame)
keyboard_v_box.addWidget(bot_frame)
keyboard_frame = QFrame()
keyboard_frame.setLayout(keyboard_v_box)
# Create main layout and add widgets
main_v_box = QVBoxLayout()
main_v_box.addWidget(self.hangman_label)
main_v_box.addWidget(self.word_frame)
main_v_box.addWidget(keyboard_frame)
# Create central widget for main window
central_widget = QWidget()
central_widget.setLayout(main_v_box)
self.setCentralWidget(central_widget)
def buttonPushed(self, button):
    """
    Handle buttons from the button group and game logic.
    """
    button.setEnabled(False)
    body_parts_list = ["head", "body", "right_arm",
                      "left_arm", "right_leg", "left_leg"]
    # When the user guesses incorrectly and the number of incorrect
    # turns is not equal to 6 (the number of body parts).
    if button.text() not in self.chosen_word and self.hangman_label.incorrect_turns <= 5:
        self.hangman_label.incorrect_turns += 1
        index = self.hangman_label.incorrect_turns - 1
        self.hangman_label.part_list.append(body_parts_list[index])

```

```

        self.hangman_label.incorrect_letter = True
    # When a correct letter is chosen, update labels and
    # correct counter.
    elif button.text() in self.chosen_word and self.hangman_label.incorrect_turns <= 5:
        self.hangman_label.incorrect_letter = True
        for i in range(len(self.chosen_word)):
            if self.chosen_word[i] == button.text():
                self.labels[i].setText(button.text())
                self.correct_counter += 1
    # Call update before checking winning conditions
    self.update()
    # User wins when the number of correct letters equals
    # the length of the word.
    if self.correct_counter == len(self.chosen_word):
        self.displayDialogs("win")
    # Game over if number of incorrect turns equals
    # the number of body parts. Reveal word to user.
    if self.hangman_label.incorrect_turns == 6:
        for i in range(len(self.chosen_word)):
            self.labels[i].setText(self.chosen_word[i])
        self.displayDialogs("game_over")
def openFile(self):
    """
    Open words.txt file.
    """
    try:
        with open("files/words.txt", 'r') as f:
            word_list = f.read().splitlines()
            return word_list
    except FileNotFoundError:
        print("File Not Found.")
        ex_list = ["nofile"]
        return ex_list
def displayDialogs(self, text):
    """
    Display win and game over dialog boxes.
    """
    if text == "win":
        message = QMessageBox().question(self, "Win!",
            "You Win!\nNEW GAME?", QMessageBox.Yes | QMessageBox.No, QMessageBox.No)
    elif text == "game_over":
        message = QMessageBox().question(self, "Game Over",
            "Game Over\nNEW GAME?", QMessageBox.Yes | QMessageBox.No, QMessageBox.No)
    if message == QMessageBox.No:
        self.close()
    else:
        self.newGame()
if __name__ == '__main__':
    app = QApplication(sys.argv)
    app.setStyleSheet(style_sheet)
    window = Hangman()
    sys.exit(app.exec_())

```

Listing 12-5 This is the code for the Hangman GUI

The finished hangman GUI can be seen in Figure [12-6](#).

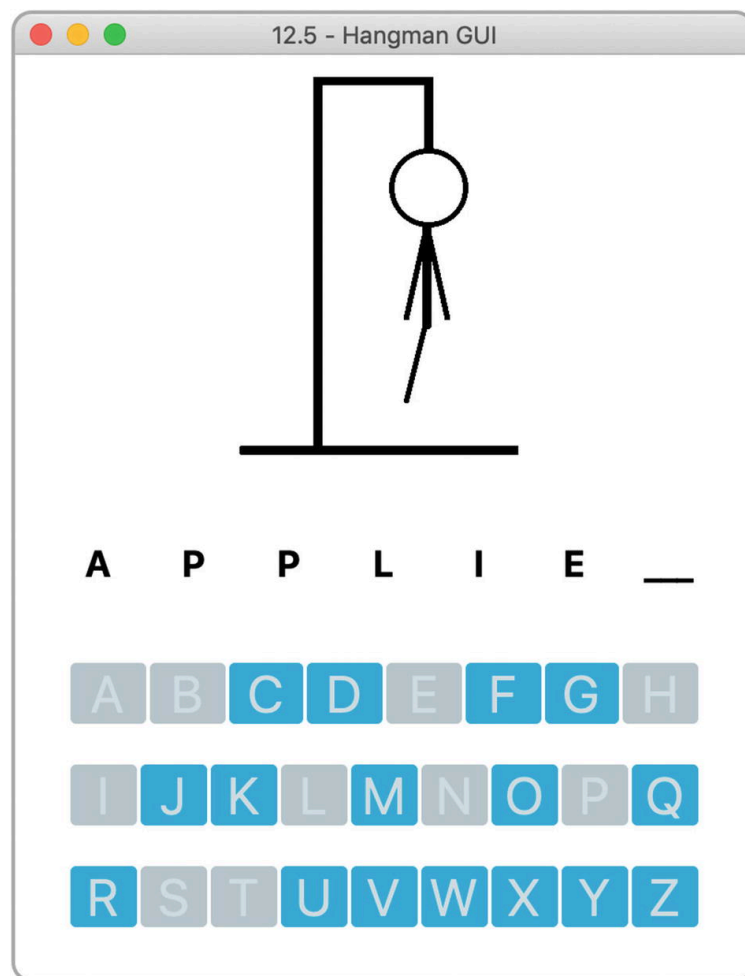


Figure 12-6 The Hangman application. Can you save him?

Explanation

A variety of classes are used in the Hangman GUI, including different widgets from `QtWidgets`, as well as classes used for drawing from `QtCore` and `QtGui`. We then create a style sheet to set the style properties of the widgets and to handle the situation of how the buttons look when they are pressed.

This program contains two classes, `DrawingLabel` and `Hangman`.

Creating the Drawing Class

The `DrawingLabel` class inherits from `QLabel` and handles the different paint events that will be drawn on the label object in the main window. The `paintEvent()` function is called in a class that inherits from `QLabel` so that way the paint events occur on the label and are not covered up by the main window.

In order to use this class, an instance is created in the `Hangman` class:

```
self.hangman_label = DrawingLabel(self)
```

The `paintEvent()` function sets up `QPainter` and handles the two painting methods, `drawHangmanBackground()`, which draws the gallows of the

Hangman game onto the label, and `drawHangmanBody()`, which only draws the body parts if they are contained in the `part_list`.

Creating the Main Window Class

The Hangman class starts by initializing the GUI window and calling the `newGame()` method. First, the Hangman board is created as an instance of the `DrawingLabel` class. Then, `setupBoard()` selects a random word from the `words.txt` file. The labels that will represent the letters of the chosen word are replaced with underscore characters, appended to the `labels` list, and added to the horizontal layout of the `word_frame` object.

Finally, we need to set up the keyboard push buttons, layouts, and the game logic in `setupBoard()`. Three rows of push buttons that represent the letters of the alphabet are controlled by one `QButtonGroup` object, `keyboard_bg`. When one button is pushed, it generates a signal that calls the `buttonPushed` slot.

When a push button is pressed, it is disabled.

```
button.setEnabled(False)
```

The list of body parts contains the six body part names. If the player guesses an incorrect letter, the name is appended to the `part_list` and checked for in the `drawHangmanBody()` function. Using this method ensures that all necessary parts are drawn with their different styles when `paintEvent()` is called. Otherwise, the labels are updated to display the correct letters in the appropriate positions if the player guesses correctly.

If the player wins or loses, a `QMessageBox` will appear and allow the user to close the application or continue. If `Yes` is selected, `newGame()` is called.

Project 12.6 – Web Browser GUI

A web browser is a graphical user interface that allows access to information on the World Wide Web (Listing 12-6). A user can enter a Uniform Resource Locator (URL) into an address bar and request content for a web site from a web server to be displayed on their local device, including text, image, and video data. URLs are generally prefixed with `http`, a protocol used for fetching and transmitting requested web pages, or `https`, for encrypted communication between browsers and web sites.

Qt provides quite a few classes for network communication, `WebSockets`, support for accessing the World Wide Web, and more. This project introduces PyQt's classes for web integration into GUIs.

For the following project, we will take a look at `QtWebEngine`, specifically the `QtWebEngineWidgets` module for creating widget-based web applications. `QtWebEngine` provides a web browser engine that can be used to embed web content into your applications. The `QtWebEngine` module

uses Chromium as its back end. Chromium is open source software from Google that can be used to create web browsers.

The web browser GUI serves as a framework for creating your own web browser and includes the following features:

- Ability to open multiple windows and tabs, either by using the application's menu or shortcut hot keys
- A navigation bar that is made up of back, forward, refresh, stop, and home buttons and the address bar for entering URLs
- The web engine view widget created using `QWebEngineView`
- A status bar
- A progress bar that relays feedback to the user about loading web pages

Note When you are running this program, if you get an error message stating “No module named: PyQt5.QtWebEngineWidgets”, then you need to install the `QtWebEngineWidgets` module. To solve this problem, enter the following command into the command line: `pip3 install PyQtWebEngine`.

```
# web_browser.py
# Import necessary modules
import os, sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QLabel, QLineEdit, QDesktopWidget, QTabWidget)
from PyQt5.QtCore import QSize, QUrl
from PyQt5.QtGui import QIcon
from PyQt5.QtWebEngineWidgets import QWebEngineView
style_sheet = """
    QTabWidget:pane{
        border: none
    }
"""

class WebBrowser(QMainWindow):
    def __init__(self):
        super().__init__()
        # Create lists that will keep track of the new windows,
        # tabs, and URLs
        self.window_list = []
        self.list_of_web_pages = []
        self.list_of_urls = []
        self.initializeUI()
    def initializeUI(self):
        self.setMinimumSize(300, 200)
        self.setWindowTitle("12.6 - Web Browser")
        self.setWindowIcon(QIcon(os.path.join('images', 'pyqt_logo.png')))
        self.positionMainWindow()
        self.createMenu()
        self.createToolBar()
        self.createTabs()
        self.show()
    def createMenu(self):
        """
        Set up the menu bar.
        """
        new_window_act = QAction('New Window', self)
        new_window_act.setShortcut('Ctrl+N')
```

```

new_window_act.triggered.connect(self.openNewWindow)
new_tab_act = QAction('New Tab', self)
new_tab_act.setShortcut('Ctrl+T')
new_tab_act.triggered.connect(self.openNewTab)
quit_act = QAction("Quit Browser", self)
quit_act.setShortcut('Ctrl+Q')
quit_act.triggered.connect(self.close)
# Create the menu bar
menu_bar = self.menuBar()
menu_bar.setNativeMenuBar(False)
# Create file menu and add actions
file_menu = menu_bar.addMenu('File')
file_menu.addAction(new_window_act)
file_menu.addAction(new_tab_act)
file_menu.addSeparator()
file_menu.addAction(quit_act)
self.status_bar = QStatusBar()
self.setStatusBar(self.status_bar)
def createToolbar(self):
    """
    Set up the navigation toolbar.
    """
    tool_bar = QToolBar("Address Bar")
    tool_bar.setIconSize(QSize(30, 30))
    self.addToolBar(tool_bar)
    # Create toolbar actions
    back_page_button = QAction(QIcon(os.path.join('icons', 'back.png')), "Back", self)
    back_page_button.triggered.connect(self.backPageButton)
    forward_page_button = QAction(QIcon(os.path.join('icons', 'forward.png')), "Forward", self)
    forward_page_button.triggered.connect(self.forwardPageButton)
    refresh_button = QAction(QIcon(os.path.join('icons', 'refresh.png')), "Refresh", self)
    refresh_button.triggered.connect(self.refreshButton)
    home_button = QAction(QIcon(os.path.join('icons', 'home.png')), "Home", self)
    home_button.triggered.connect(self.homeButton)
    stop_button = QAction(QIcon(os.path.join('icons', 'stop.png')), "Stop", self)
    stop_button.triggered.connect(self.stopButton)
    # Set up the address bar
    self.address_line = QLineEdit()
    # addAction() is used here to merely display the icon in the line edit
    self.address_line.addAction(QIcon('icons/search.png'), QLineEdit.LeadingPosition)
    self.address_line.setPlaceholderText("Enter website address")
    self.address_line.returnPressed.connect(self.searchForUrl)
    tool_bar.addAction(home_button)
    tool_bar.addAction(back_page_button)
    tool_bar.addAction(forward_page_button)
    tool_bar.addAction(refresh_button)
    tool_bar.addWidget(self.address_line)
    tool_bar.addAction(stop_button)
def createTabs(self):
    """
    Create the QTabWidget object and the different pages.
    Handle when a tab is closed.
    """
    self.tab_bar = QTabWidget()
    self.tab_bar.setTabsClosable(True) # Add close buttons to tabs
    self.tab_bar.setTabBarAutoHide(True) # Hides tab bar when less than 2 tabs
    self.tab_bar.tabCloseRequested.connect(self.closeTab)
    # Create tab
    self.main_tab = QWidget()
    self.tab_bar.addTab(self.main_tab, "New Tab")
    # Call method that sets up each page

```

```

self.setupTab(self.main_tab)
self.setCentralWidget(self.tab_bar)
def setupWebView(self):
    """
    Create the QWebEngineView object that is used to view
    web docs. Set up the main page, and handle web_view signals.
    """
    web_view = QWebEngineView()
    web_view.setUrl(QUrl("https://google.com"))
    # Create page loading progress bar that is displayed in
    # the status bar.
    self.page_load_pb = QProgressBar()
    self.page_load_label = QLabel()
    web_view.loadProgress.connect(self.updateProgressBar)
    # Display URL in address bar
    web_view.urlChanged.connect(self.updateUrl)
    ok = web_view.loadFinished.connect(self.updateTabTitle)
    if ok:
        # Web page loaded
        return web_view
    else:
        print("The request timed out.")
def setupTab(self, tab):
    """
    Create individual tabs and widgets. Add the tab's url and web view to the appropriate list.
    Update the address bar if the user switches tabs.
    """
    # Create the web view that will be displayed on the page.
    self.web_page = self.setupWebView()
    tab_v_box = QVBoxLayout()
    # Sets the left, top, right, and bottom margins to use around the layout.
    tab_v_box.setContentsMargins(0,0,0,0)
    tab_v_box.addWidget(self.web_page)
    # Append new web_page and URL to the appropriate lists
    self.list_of_web_pages.append(self.web_page)
    self.list_of_urls.append(self.address_line)
    self.tab_bar.setCurrentWidget(self.web_page)
    # If user switches pages, update the URL in the address to
    # reflect the current page.
    self.tab_bar.currentChanged.connect(self.updateUrl)
    tab.setLayout(tab_v_box)
def openNewWindow(self):
    """
    Create new instance of the WebBrowser class.
    """
    new_window = WebBrowser()
    new_window.show()
    self.window_list.append(new_window)
def openNewTab(self):
    """
    Create new tabs.
    """
    new_tab = QWidget()
    self.tab_bar.addTab(new_tab, "New Tab")
    self.setupTab(new_tab)
    # Update the tab_bar index to keep track of the new tab.
    # Load the URL for the new page.
    tab_index = self.tab_bar.currentIndex()
    self.tab_bar.setCurrentIndex(tab_index + 1)
    self.list_of_web_pages[self.tab_bar.currentIndex()].load(QUrl("https://google.com"))
def updateProgressBar(self, progress):

```

```

"""
Update progress bar in status bar.
This provides feedback to the user that the page is still loading.
"""

if progress < 100:
    self.page_load_pb.setVisible(progress)
    self.page_load_pb.setValue(progress)
    self.page_load_label.setVisible(progress)
    self.page_load_label.setText("Loading Page... ({}/100)".format(str(progress)))
    self.status_bar.addWidget(self.page_load_pb)
    self.status_bar.addWidget(self.page_load_label)
else:
    self.status_bar.removeWidget(self.page_load_pb)
    self.status_bar.removeWidget(self.page_load_label)
def updateTabTitle(self):
    """
    Update the title of the tab to reflect the website.
    """
    tab_index = self.tab_bar.currentIndex()
    title = self.list_of_web_pages[self.tab_bar.currentIndex()].page().title()
    self.tab_bar.setTabText(tab_index, title)
def updateUrl(self):
    """
    Update the url in the address to reflect the current page being displayed.
    """
    url = self.list_of_web_pages[self.tab_bar.currentIndex()].page().url()
    formatted_url = QUrl(url).toString()
    self.list_of_urls[self.tab_bar.currentIndex()].setText(formatted_url)
def searchForUrl(self):
    """
    Make a request to load a url.
    """
    url_text = self.list_of_urls[self.tab_bar.currentIndex()].text()
    # Append http to URL
    url = QUrl(url_text)
    if url.scheme() == "":
        url.setScheme("http")
    # Request URL
    if url.isValid():
        self.list_of_web_pages[self.tab_bar.currentIndex()].page().load(url)
    else:
        url.clear()
def backPageButton(self):
    tab_index = self.tab_bar.currentIndex()
    self.list_of_web_pages[tab_index].back()
def forwardPageButton(self):
    tab_index = self.tab_bar.currentIndex()
    self.list_of_web_pages[tab_index].forward()
def refreshButton(self):
    tab_index = self.tab_bar.currentIndex()
    self.list_of_web_pages[tab_index].reload()
def homeButton(self):
    tab_index = self.tab_bar.currentIndex()
    self.list_of_web_pages[tab_index].setUrl(QUrl("https://google.com"))
def stopButton(self):
    tab_index = self.tab_bar.currentIndex()
    self.list_of_web_pages[tab_index].stop()
def closeTab(self, tab_index):
    """
    This signal is emitted when the close button on a tab is clicked.
    The index is the index of the tab that should be removed.
    """

```



```

"""
self.list_of_web_pages.pop(tab_index)
self.list_of_urls.pop(tab_index)
self.tab_bar.removeTab(tab_index)
def positionMainWindow(self):
"""
Use QDesktopWidget class to access information about your screen
and use it to position the application window when starting a new application.
"""

desktop = QDesktopWidget().screenGeometry()
screen_width = desktop.width()
screen_height = desktop.height()
self.setGeometry(0, 0, screen_width, screen_height)
if __name__ == '__main__':
    app = QApplication(sys.argv)
    app.setStyleSheet(style_sheet)
    window = WebBrowser()
    app.exec_()

```

Listing 12-6 Web browser GUI code

Your application should look similar to Figure [12-7](#).

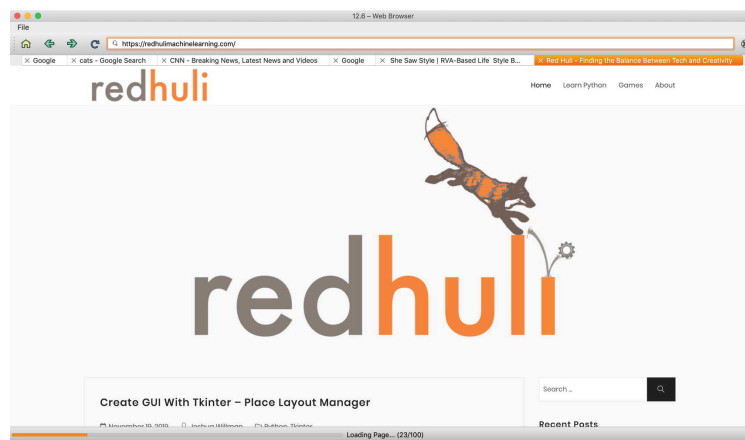


Figure 12-7 The web browser GUI displaying the menubar, toolbar, different tabs, the logo for my blog, RedHuli, and the progress bar at the bottom

Explanation

Two new classes are introduced in this application – **QUrl** is used for managing and constructing URLs, and **QWebEngineView** is used for creating the main component for rendering content from the Web, the web engine view (denoted as `web_view` in the code).

Before calling `initializeUI()`, we need to instantiate a few lists that will contain the new windows, web pages viewed, and URLs for each tab. This project also calls `setWindowIcon()` to include an application icon, but it will not be displayed on MacOS due to system guidelines.

There are three main methods that are called in `initializeUI()`. The first one is `createMenu()` for setting up the main menu and the status bar. The menu includes actions and shortcuts for creating new windows, new tabs, and closing the application.

Next is the `createToolbar()` method that creates the navigation bar of the web browser. The `tool_bar` includes buttons for navigating between web pages

and a `QLineEdit` widget for entering and displaying URLs. Each button emits a signal when triggered that is connected to an appropriate slot. For example, if the `back_page_button` is pressed, the `backPageButton()` slot will be called.

```
def backPageButton(self):
    tab_index = self.tab_bar.currentIndex()
    self.list_of_web_pages[tab_index].back()
```

The current index of the tab we are viewing is stored in `tab_index`. The `back()` method is then called on the `web_view` object for that current tab. If the `tab_index` is not 0, then the user can navigate back through previously viewed web pages. The `back()` method is but one of several functions included in the `QWebEngineView` class. Other methods for navigation include `forward()`, `reload()`, and `stop()`, and these are also utilized for the other `tool_bar` buttons.

When the user enters a web address in the `QLineEdit` widget and presses the return key, we check to see if the URL begins with the correct scheme (such as `http`, `https`, or `file`). If a valid scheme is not present, `http` is appended to the beginning of the URL. If the URL conforms to standard encoding rules, a request is then sent to `load()` the web site.

Creating Tabs for the Web Browser

The third method, `createTabs()`, is used to handle creating the tab widget and the web view objects. First, we need to create the `QTabWidget` that will display each individual tab's web view. Refer back to Chapter 6 for more details on setting up tab widgets.

A few of the `tab_bar` widget's parameters are changed so that each tab includes a close button, and if only one tab remains, then the tab bar will not be displayed. This helps to make sure that there is always at least one tab in the main window. If a tab is closed, the `closeTab()` slot is called. The corresponding URLs and web view items for that tab are also removed from the `list_of_urls` and `list_of_web_pages` lists.

The first tab, `main_tab`, is created, added to the `tab_bar`, and then passed to the `setupTab()` method. The `tab_bar` widget is set as the central widget for the main window. When setting up a tab's page, we first need to create a web view object.

Creating the Web View

The `setupWebView()` function creates an instance of the `QWebEngineView` class, `web_view`, and sets the web view's URL to display the Google web page.

```
web_view.setUrl(QUrl("https://google.com"))
```

To create a basic instance of a web view in an application, you only need to create a `QWebEngineView` object, use the `load()` method to load the web page onto the web view widget, and then call `show()`. The following code shows the process for setting up a simple web view widget:

```
web_view = QWebEngineView()
```

```
web_view.load(QUrl("https://google.com"))
web_view.show()
```

Once the web page has loaded, the `urlChanged()` signal connected to `updateUrl()` changes the URL displayed in `address_line`. We can use the `loadFinished()` signal to tell the current tab to update its title using the `updateTabTitle()` slot and return the `web_view` widget.

Next, create the layout to hold the web view widget, append the current tab's URL and `web_page` object to the `list_of_urls` and `list_of_web_pages` lists, and set the layout for the current tab's page. The `web_page` object is the `web_view` widget that is returned from `setupWebView()` and displayed in the page of the tab.

Finally, to handle when a user switches between tabs, `QTabWidget` has the `currentChanged()` signal. If a different tab is selected, the connected slot, `updateUrl()`, will change the displayed URL in `address_line`.

Adding a QProgressBar to the Status Bar

In `setupWebView()`, a progress bar and label are also created that will be used to display the loading progress of a web page in the browser's status bar. When the `loadProgress()` signal is generated, the `updateProgressBar()` slot is called.

```
web_view.loadProgress.connect(self.updateProgressBar)
```

The `loadProgress()` function returns an `int` value that we can use to track how much of the page has loaded. While `progress` is less than 100, the progress bar and the label are both displayed and their values are set. The code for displaying the progress bar is shown as follows:

```
self.page_load_pb.setVisible(progress)
self.page_load_pb.setValue(progress)
```

The widgets are then added to the status bar.

```
self.status_bar.addWidget(self.page_load_pb)
```

When a page is finished loading, we call `removeWidget()` to remove the progress bar and the label. An example of the progress bar can be seen at the bottom of Figure [12-7](#).

Note Creating a web browser is a very extensive task. There are many topics that are not included in this project, such as accessing HTTP cookies with `QtWebEngineCore`, working with the browser history with `QWebEngineHistory`, managing connections and client certificates, proxy support with `QNetworkProxy`, working with JavaScript, downloading content from web sites, and others. You are definitely encouraged to research these topics if you need to use `QtWebEngine` for more advanced projects.

Summary

In this chapter, you saw different GUI applications that build the structure for larger projects, such as the camera GUI or the web browser GUI. Other projects introduced components that you may be able to include in other programs, such as the directory viewer GUI, the clock GUI, or the calendar GUI. In the case of the Hangman GUI, a complete program was created so that it can hopefully give you ideas for other programs you may want to design.

We have explored a variety of topics for designing graphical user interfaces using PyQt5 and Python throughout this book – different types of widgets, classes, and layouts. We saw how to stylize your interfaces, how to add menus, and how to make creating an application simpler with Qt Designer.

We covered a few advanced topics such as working with the clipboard, SQL, and multithreaded applications, as well.

Appendix A will fill in more details about the PyQt5 classes used in this book, as well as a few other classes that there was no room to include in the previous chapters.

Appendix B is used to refresh your knowledge on the key Python concepts used in this book.

Your feedback and questions are always welcome. Thank you so much for traveling along with me on this journey to create this guide for you.

Footnotes

¹ <https://doc.qt.io/qt-5/qcameraviewfinder.html>

² <https://doc.qt.io/qt-5/qtwidgets-widgets-calendarwidget-example.html>
